BVS1 – Betriebssysteme

Wintersemester 2026/2027 · TH Köln · ICCT

Stavros Kousidis

Table of contents

Course Overview 3
Course Material . . . 3

1 From Executing Code to Operating Systems 4

2 From Executing Code to Operating Systems 5
2.1 1. One program can run without a guest operating system . . . 5
2.1.1 Interpreting the experiment . . . 6
2.2 2. The ISA connects software and processor . . . 7
2.2.1 Why RV64? . . . 7
2.3 3. Development host and target are different roles . . . 7
2.4 4. Virtualization and emulation are different . . . 8
2.4.1 Virtualization: the guest instructions already belong to the host ISA . . . 8
2.4.2 Emulation: the target instruction may not be a host instruction . . . 9
2.4.3 Our BVS1 machine stack . . . 9
2.5 5. The toolchain connects development host and target . . . 10
2.5.1 Object files are not yet the complete program . . . 11
2.6 6. When does an operating system become useful? . . . 11
2.7 7. Applying the working definition . . . 12
2.8 8. From Hello BVS1 to a minimal operating system . . . 12
2.9 9. Key ideas . . . 14
2.10 References . . . 14

3 From Source to Process 16
3.1 1. From source code to target-specific object code . . . 16
3.2 2. Why object files still need a linker . . . 17
3.3 3. From the stored-program idea to a concrete memory layout . . . 17
3.4 4. Linking produces the program QEMU can load . . . 18
3.5 5. Why execution starts before `main` . . . 19
3.6 6. The whole path, and what Make hides . . . 19
3.7 7. Change, rebuild, observe . . . 20
3.8 8. Why an executing program is not yet a process . . . 20
3.9 9. Yield, exit, resume, and restart . . . 21
3.10 10. The question that leads to execution context . . . 22

Course Overview

This site contains the teaching material for **BVS1 – Betriebssysteme** at TH Köln, ICCT, for **Wintersemester 2026/2027 · TH Köln · ICCT**. The course is attended by students from multiple Bachelor programmes.

The course provides two aligned representations: concise lecture-specific Beamer slides for classroom interaction, and a reader-oriented chapter source used for both the PDF script and the HTML course.

Course Material

The lecture chapters are listed in the HTML navigation and in the PDF table of contents.

1 From Executing Code to Operating Systems

2 From Executing Code to Operating Systems

An operating system is familiar enough that it is easy to begin with examples rather than with the problem it solves. Windows, Ubuntu, Android and macOS are recognizable operating-system environments, but naming familiar systems does not yet explain why operating systems exist.

BVS1 therefore starts with a smaller question:

Can a computer execute a program without an operating system?

The first practical system is deliberately tiny. An RV64 target machine runs one program in QEMU and prints:

```
Hello BVS1
```

The experiment gives us a concrete machine to reason about before we introduce operating-system mechanisms.

2.1 1. One program can run without a guest operating system

At the lowest level, a processor executes instructions. Those instructions operate on registers and memory and can communicate with devices.

One important idea of the stored-program / Von Neumann model is that **instructions and data are both kept in memory**.

A deliberately simplified machine model is:

```
     instructions + data


          memory


            CPU


          devices
```

This model omits many details of real hardware, but it is enough for the first distinction:

> **CPU, memory and devices exist independently of an operating system.**

The point here is conceptual rather than historical or microarchitectural. Modern processors may use separate instruction and data caches internally, but the important point here is that instructions and data both occupy target memory. Lecture 2 returns to this idea when the linker script makes the abstract memory model concrete.

In the `Hello BVS1` experiment, QEMU provides the target machine. The target contains an RV64 CPU, memory and devices. The program is loaded and begins executing without a guest operating system.

The physical computer still runs an operating system, and our Ubuntu development environment also runs an operating system. The important observation concerns the **target**:

> **Executing instructions is not, by itself, what makes an operating system necessary.**

2.1.1 Interpreting the experiment

The classroom experiment deliberately separates observation from explanation. Consider these statements:

- QEMU executed our program.
- A virtual CPU executed our program.
- Ubuntu executed our guest program.
- The target had an operating system.
- QEMU was already running before our guest program started.
- The string `Hello BVS1` existed somewhere in memory while the program was running.

The purpose of discussing them is to expose the execution model.

A precise description is:

- the Ubuntu development VM runs the QEMU process;
- QEMU implements the target machine;
- the target contains an emulated RV64 CPU, memory and devices;
- the RV64 program runs on that target;
- the target has no guest operating system.

Saying "QEMU executed the program" is understandable in informal language, but the more useful model is that QEMU **emulates the machine whose CPU executes the target instructions**.

2.2 2. The ISA connects software and processor

Machine code only has meaning if software and the processor agree on what each instruction means.

This agreement is the **Instruction Set Architecture (ISA)**.

An ISA specifies properties visible to machine-level software, including:

- instructions and their encodings,
- registers,
- memory-addressing rules,
- exceptional control transfers,
- privilege mechanisms.

Different processor implementations can implement the same ISA.

The ISA should therefore be distinguished from the **microarchitecture**:

- **ISA:** the contract visible to software;
- **microarchitecture:** how a particular processor implements that contract internally.

For BVS1, the teaching target uses **RV64**, the 64-bit form of RISC-V.

2.2.1 Why RV64?

RISC-V is a teaching choice, not a claim that it is the best ISA for every system.

It offers a comparatively regular architecture, public specifications, a modern privilege model, suitable development tools, and a well-supported QEMU `virt` machine. These properties let us expose operating-system mechanisms without making historical processor complexity the subject of the course.

2.3 3. Development host and target are different roles

The machine on which software is developed does not need to use the same ISA as the machine for which the program is built.

In BVS1:

- the **development host** is the Ubuntu `bvs1` VM in which the development tools run;
- the **target** is the RV64 machine for which we build the program.

The development VM may itself run on an x86-64 or Arm64 host platform. The teaching target is nevertheless RV64.

This gives two separate questions:

1. Where do the development tools execute?
2. Which machine will execute the generated program?

Keeping these roles separate is essential for understanding both cross-compilation and emulation.

2.4 4. Virtualization and emulation are different

The most useful first question is:

Can the physical CPU execute the guest or target instructions directly?

That question separates the roles of virtualization and emulation more clearly than the word *virtual machine* by itself.

2.4.1 Virtualization: the guest instructions already belong to the host ISA

In hardware-assisted virtualization, guest and host normally use the same processor ISA.

A guest instruction is therefore, in the ordinary case, already an instruction that the physical CPU understands. Most non-privileged guest instructions can execute directly on the physical processor.

The virtualization mechanism controls the parts that must not simply be handed to the guest, for example:

- isolation between machines,
- privileged processor state,
- access to memory and devices,
- transitions that require intervention by the virtualization layer.

A simplified model is:

```
guest instruction
       ↓
physical CPU
```

The important point is not that virtualization performs no mediation. It is that the guest machine does not require a different instruction vocabulary from the physical CPU.

In BVS1, Multipass manages the Ubuntu development VM using virtualization facilities available on the host platform. The exact backend can differ between operating systems.

2.4.2 Emulation: the target instruction may not be a host instruction

Emulation can reproduce a machine whose ISA differs from the host ISA.

Suppose the physical processor uses x86-64 while the BVS1 target uses RV64. The physical x86-64 CPU does not directly understand an RV64 instruction.

An emulator must therefore reproduce the effect of the target instruction. Conceptually:

```
RV64 target instruction
          ↓
        QEMU
          ↓
interpret / translate
          ↓
host operations
          ↓
physical CPU
```

The word **interpret** is useful for the conceptual distinction, but it should not be understood as a claim that an emulator must process every instruction individually. QEMU can dynamically translate blocks of target instructions into host code.

The essential point is:

> **Virtualization can largely run guest instructions directly; emulation must reproduce what target instructions would do when the host cannot execute them directly.**

2.4.3 Our BVS1 machine stack

The resulting stack is:

```
physical host
     ↓  virtualization (managed through Multipass)
Ubuntu development VM: bvs1
     ↓  runs
QEMU process
     ↓  emulates RV64
RV64 target machine
     ↓  executes
Hello BVS1
```

The layers have different roles:

- the **physical host** provides the real CPU, memory and devices;

- the **Ubuntu development VM** is the common environment in which the course tools run;
- **Multipass** manages that development VM and is not an additional machine layer;
- **QEMU** is a process running inside the development VM;
- QEMU emulates the **RV64 target machine**;
- `Hello BVS1` executes on that target.

Only the RV64 target is the teaching machine whose operating-system mechanisms we will build. It initially contains no guest operating system.

A **virtual machine** is the machine environment presented to software. Virtualization and emulation are different techniques that can provide such environments.

QEMU itself is capable of more than the configuration used here: in other settings it can also participate in hardware-assisted virtualization. In BVS1, its relevant role is RV64 emulation.

2.5 5. The toolchain connects development host and target

The source program is written in a human-readable programming language, but the target CPU needs RV64 machine instructions.

A simplified build path is:

```
source code
    ↓
cross compiler / assembler
    ↓
object files
    ↓
linker
    ↓
RV64 executable
    ↓
QEMU
    ↓
RV64 target machine
```

The cooperating compiler, assembler, linker and binary utilities form the **toolchain**.

A **cross compiler** runs on one architecture while producing code for another target architecture.

QEMU has a different role: it does not translate source code into the executable. It provides the machine on which the already generated target program can run.

2.5.1 Object files are not yet the complete program

Compilation and assembly can already produce target-specific object files. These can contain RV64 machine code, but they are not yet the complete executable. Separate pieces still have to be combined and references between them resolved.

The final result of this build path is a linked RV64 ELF executable that QEMU can load for the target machine.

Lecture 2 unfolds this path manually: we inspect the object files, discover what is still unresolved, and then see why linking and the target memory layout are necessary.

2.6 6. When does an operating system become useful?

The `Hello BVS1` target can run one tiny program without a guest operating system.

Now change the requirement:

> **The machine must run two programs.**

New questions immediately appear:

- Which program receives the processor?
- Where is each program's state kept?
- What happens when one program waits for a device?
- Can one program overwrite another program's memory?
- Who coordinates access to shared devices?
- How does a program request a shared service?

Increase the requirements further:

- many programs,
- multiple users,
- persistent storage,
- keyboard and display,
- networking,
- failures,
- programs that do not trust one another.

The problem is no longer merely "execute instructions".

Now the system needs mechanisms for **sharing, coordination, protection and abstraction**.

This gives us the working definition used in BVS1:

> An **operating system** is software that helps turn hardware into a usable computer system by providing abstractions, resource management, coordination, protection, and services for programs and users.

This is intentionally a working definition.

We will test and sharpen it as concrete mechanisms are introduced.

Two classical viewpoints are already visible:

1. **Operating system as resource manager**
   CPU time, memory, storage and devices are finite resources that must be shared and controlled.
2. **Operating system as abstract machine**
   Programs should not need to manipulate every hardware detail directly. The operating system can provide safer and more useful abstractions and services.

2.7 7. Applying the working definition

The opening lecture deliberately asks students to classify several familiar terms before the working definition is established.

After deriving the definition, the distinctions become clearer:

- **Windows, Ubuntu and Android** are complete operating-system environments.
- The **Linux kernel** is the core used by Linux-based operating-system environments, not such an environment by itself.
- **UEFI** is firmware and a firmware interface/environment.
- **QEMU** is an emulator and virtualizer.
- **Docker** provides container tooling built on operating-system mechanisms.
- **Firefox** is an application.

The purpose of the exercise is not to memorize labels.

It is to ask which responsibilities a component actually assumes and at which layer it operates.

Exercise 1 extends this idea with less obvious boundary cases such as a boot manager, Proxmox, and the deliberately provocative question “Is Emacs an operating system?”

2.8 8. From Hello BVS1 to a minimal operating system

The course deliberately starts with a target that can do very little:

```
Hello BVS1
```

The semester question is:

> **What must we add to Hello BVS1 before we would call the result an operating system?**

Initial answers may include:

- handling devices,
- multiple executions,
- scheduling,
- protection,
- memory management,
- system services,
- communication,
- persistent storage,
- users,
- a command interface.

At the beginning these are hypotheses, not a checklist to memorize.

We add mechanisms only when a new requirement gives us a reason for them.

The practical target remains:

> **a small educational operating system on RV64 and QEMU `virt`, eventually providing a simple shell**

The red thread is:

```
a machine executes code
    ↓
software communicates with devices
    ↓
events must be handled
    ↓
multiple programs must coexist
    ↓
programs compete for the CPU
    ↓
programs and resources need protection
    ↓
programs need operating-system services
    ↓
programs need coordination
    ↓
programs need communication
    ↓
a user needs to control programs
    ↓
a simple shell
```

The course is therefore not organized as a catalogue of terminology.

Each new mechanism should answer a problem that the current system has acquired.

2.9 9. Key ideas

Lecture 1 establishes a small number of distinctions that we will use repeatedly:

- A target processor can execute a program without a guest operating system.
- CPU, memory and devices exist independently of the OS.
- An ISA defines the machine-level contract on which software relies.
- Development host and target are different roles.
- Virtualization and emulation are different techniques.
- Multipass gives us the virtualized development environment; QEMU emulates our RV64 target in the BVS1 setup.
- A cross-toolchain produces machine code for a target ISA different from the development system.
- Operating systems become useful as requirements for sharing, abstraction, coordination, protection and services grow.
- The BVS1 system grows only when a new requirement motivates a new mechanism.

A useful test of understanding is to explain this complete path:

```
source code
    ↓
cross-toolchain
    ↓
RV64 executable
    ↓
QEMU process in the development VM
    ↓
emulated RV64 machine
    ↓
observable program behaviour
```

and then explain why successful execution still does **not** imply that the target contains an operating system.

2.10 References

- Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*. https://pages.cs.wisc.edu/~remzi/OSTEP/
- Andrew S. Tanenbaum and Herbert Bos, *Modern Operating Systems*, Pearson.

- RISC-V International, RISC-V ISA specifications. https://docs.riscv.org/
- QEMU documentation, RISC-V `virt` Generic Virtual Platform. https://www.qemu.org/docs/master/system/riscv/virt.html
- Canonical Multipass documentation. https://documentation.ubuntu.com/multipass/

3 From Source to Process

Lecture 1 showed that an RV64 target can execute a small program without a guest operating system. Lecture 2 asks how that execution comes into existence and why, for an operating system, simply starting a program is not enough.

The path is:

```
source → objects → linked ELF → memory → _start → main
```

and then:

```
executing program → OS-managed execution → process
```

The build tools are therefore not the final topic. We unfold them once because they make the transition from source code to a running machine state visible.

3.1 1. From source code to target-specific object code

We first compile `main.c` without linking:

```
mkdir -p build/manual

riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -msmall-data-limit=0 -c main.c -o build/manual/main.o
```

The important option is `-c`: stop before linking.

Inspecting the result with

```
file build/manual/main.o
```

shows that `main.o` is already a RISC-V ELF object.

This distinction matters: the compiler has already translated C into code for our target ISA. The object file is therefore not architecture-neutral. But it is also not yet a complete

program. Separate pieces can still refer to one another, and their final addresses have not yet been established.

The startup assembly is translated in the same way:

```
riscv64-unknown-elf-gcc -march=rv64imac -mabi=lp64 \
  -mcmodel=medany -c start.S -o build/manual/start.o
```

3.2 2. Why object files still need a linker

Using `nm` ("name list") lets us inspect the symbols in the two objects:

```
riscv64-unknown-elf-nm build/manual/start.o
riscv64-unknown-elf-nm build/manual/main.o
```

For the symbols relevant here, we can read the result as:

```
start.o defines _start
start.o needs   main
start.o needs   _stack_top

main.o  defines main
```

The important concept is not the letters printed by `nm`, but the relationship between separately translated pieces. One object can provide a symbol while another refers to it. The linker connects those pieces.

The unresolved `_stack_top` also reveals a second responsibility of the linker: it must establish where the program lives in target memory.

3.3 3. From the stored-program idea to a concrete memory layout

Lecture 1 used the abstract model:

```
instructions + data
        ↓
      memory

       CPU
```

Our real program now contains different kinds of bytes that need concrete places:

```
RV64 memory from 0x80000000

+-----------------------------------------------------------+
| .text    _start, main, ...      | instructions          |
| .rodata  "Hello BVS1\n"         | constant data         |
| .data                           | initialized data      |
| .bss                            | zero-initialized data |
| C stack  (top: _stack_top)      | runtime data          |
+-----------------------------------------------------------+
```

These roles are not properties that memory itself knows. They arise from the executable layout and from how the program and CPU use the addresses.

The course-provided `linker.ld` makes that layout concrete:

```
ENTRY(_start)

MEMORY {
    RAM (rwx) : ORIGIN = 0x80000000, LENGTH = 128M
}

. = ALIGN(16);
. += 16K;
_stack_top = .;
```

Here `.` is the linker's current output address. After 16 KiB have been reserved for the initial stack, that address is named `_stack_top`.

Students do not need to write linker scripts in BVS1. The important idea is that the linker does not merely join files; it constructs one final address space in which the pieces can refer to one another.

3.4 4. Linking produces the program QEMU can load

We link the two objects using the course layout:

```
riscv64-unknown-elf-gcc -nostdlib -nostartfiles -no-pie \
  -T linker.ld build/manual/start.o build/manual/main.o \
  -o build/manual/hello-bvs1.elf
```

At this point the linker has:

- combined the objects;

- resolved the symbol references;
- established the final memory layout.

The resulting ELF is what QEMU receives. QEMU does not compile or link the program; it provides the emulated RV64 machine and loads the already-built image.

3.5 5. Why execution starts before `main`

The linker script declares:

```
ENTRY(_start)
```

and `_start` is defined in `start.S`:

```
_start:
    la   sp, _stack_top
    call main
1:
    j    1b
```

This small fragment exposes an important boundary between machine startup and ordinary C code.

Before `main` can run, startup code establishes at least the execution environment that C expects. Here that means setting `sp`, the stack pointer, to the address denoted by `_stack_top`.

Then `call main` transfers control to `main` and remembers where execution should return. We deliberately do not open the full calling convention yet. The fact that a return point must somehow be preserved becomes important in Lecture 3.

The local label `1:` and `1b` only express the final fallback loop: if `main` returns, execution remains there.

3.6 6. The whole path, and what Make hides

Running the ELF manually:

```
qemu-system-riscv64 -machine virt -m 128M -smp 1 -nographic \
  -bios none -kernel build/manual/hello-bvs1.elf
```

reconstructs the complete path:



Compilation, assembly, linking, and QEMU itself run in the Ubuntu development VM. The final RV64 instructions execute in the emulated target.

Make automates this workflow. It decides when a target is out of date and runs the corresponding recipe. The GCC driver performs the translation steps and invokes the linker. QEMU provides the machine on which the result executes.

Doing the path manually once is useful because the automation then has a visible meaning rather than being a black box.

3.7 7. Change, rebuild, observe

If only the string in `main.c` changes, we can predict the consequences before rebuilding:

- `main.o` must change;
- `start.o` does not;
- the final ELF must be linked again;
- `linker.ld` does not change;
- QEMU itself does not change.

This small exercise is about maintaining a causal model of the system:

predict → change → build → run → observe → explain

That pattern will be reused when the changes are no longer only strings, but operating-system mechanisms.

3.8 8. Why an executing program is not yet a process

Our current target has one execution:

```
_start
  ↓
main
  ↓
print
  ↓
for (;;) {}
```

Once it starts, nothing inside our target can currently stop it, regain control, and later continue it.

That is the new requirement. An operating system must be able to manage an execution rather than merely start it.

For our first model:

> A **process** is an execution that the operating system can manage.

For Checkpoint 1, "manage" means only:

```
create → run → yield → resume → exit
```

This is intentionally much smaller than a complete Unix process model. We do not yet need address spaces, files, process IDs, or scheduling policies.

The minimum lifecycle is:

```
create  → READY  run / resume → RUNNING  exit → FINISHED
            ↑
                  yield
```

A yielded process is still runnable. A finished process is not.

3.9 9. Yield, exit, resume, and restart

Yield and exit both transfer control back to the operating system, but they have different meanings.

- **yield**: the execution is not finished and must be able to continue later;
- **exit**: the execution is finished and must never continue.

The distinction becomes visible in this intended interface:

```
uart_puts("A\n");
process_yield();
uart_puts("B\n");
process_exit();
```

`process_yield()` and `process_exit()` are not library functions we already have. They are functions our minimal operating system still has to implement.

After `process_yield()`, the operating system has control. When the process runs again, it must continue with `uart_puts("B\n")`. Printing `A` again would mean that we restarted the execution instead of resuming it.

This is the conceptual step that makes the next lecture necessary.

3.10 10. The question that leads to execution context

To resume correctly, something about the paused execution must survive:

```
process runs
     ↓
yield
     ↓
OS regains control
     ↓
resume
     ↓
continue exactly where it stopped
```

What must be preserved so this can work?

At this point it is enough to suspect that some machine state must survive the pause. Lecture 3 determines exactly what that state is.

The final path of Lecture 2 is therefore:

```
source → objects → linked ELF → memory → _start → main
```

followed by:

```
executing program → OS-managed execution → process
```

The remaining question is:

> **What machine state makes resume possible?**